

ANUPALAN — Prototype Build Plan

Architecture, stack, dependencies and live-testing setup

Team NeuraForge · SIH26024 · AI-Based Smart Governance and Compliance Monitoring System for Coal Mines
Coal India Limited · Ministry of Coal · Software / Smart Automation · Presidency University, Bengaluru

Read this first. The stack on your deck — Kubernetes, NIC MeghRaj, Kafka, MinIO, multi-tenant isolation — is the **production** design, and it is right for the deck. It is the wrong thing to build in 36 hours. This document specifies a much thinner prototype that demonstrates the identical story end to end. When a judge asks why you didn't use Kubernetes, the answer is: "That is the deployment target. In 36 hours we built the system it deploys."

1. What the prototype must prove

scope discipline

Five demo moments. Every one maps to a claim on your deck. If a feature does not serve one of these, it does not get built.

#	DEMO MOMENT	WHAT THE JUDGE SEES	PROVES
1	Digital Rulebook	Paste a DGMS circular into a box. Seconds later new duties appear in the ledger, each citing its clause.	Regulation-as-Code. Your headline differentiator.
2	Offline capture	Phone in airplane mode. Officer photographs a duty, saves it. Turn wifi on — it syncs and appears on the dashboard.	Works underground. Evidence is geo-tagged and hash-chained.
3	Hazard early-warning	Sensor simulator pushes a methane spike. An alert fires on the dashboard, linked to the clause it breaches.	Reads existing instruments. Predicts, doesn't record.
4	Geo-compliance	Map shows the lease boundary. An excavation polygon crosses it and turns red automatically.	PostGIS boundary breach detection.
5	Auto-drafted return	System drafts a statutory return from collected evidence. Officer reviews, signs, and it locks.	Human-in-the-loop. The AI never files.

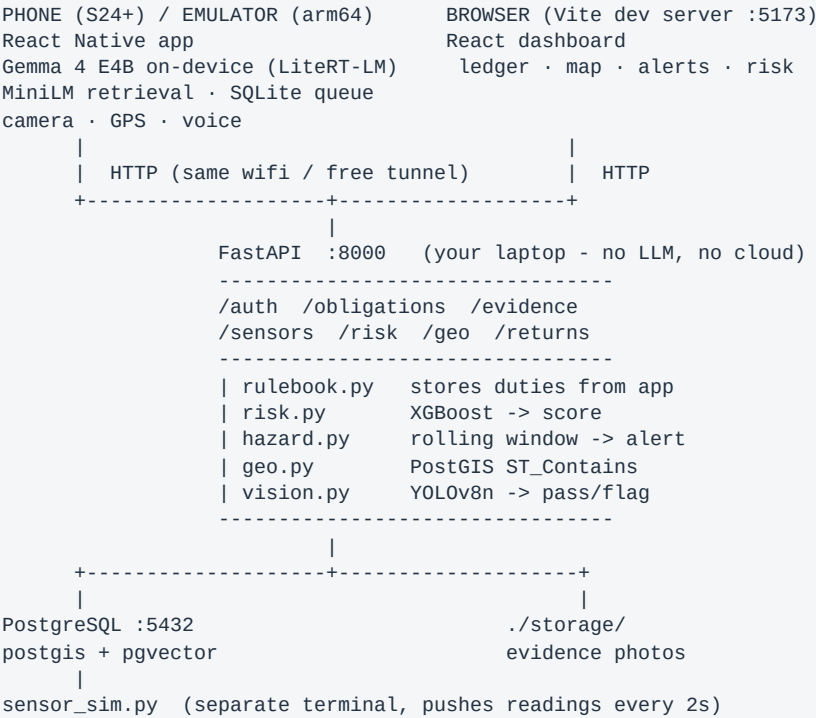
Cut list — do not build these

Kubernetes and Helm · NIC MeghRaj deployment · real Kafka cluster · MinIO (use the local filesystem) · real satellite imagery pipeline · training any model from scratch · multi-tenant subsidiary isolation · Parichay SSO · full statutory corpus (50

clauses is plenty) · production RBAC beyond three roles · unit-test coverage targets · CI/CD pipelines · dark mode · i18n beyond one demo string

2. Architecture of the prototype

four processes, one compose file



Why this shape. One database holds relational rows, spatial geometry and vector embeddings, so there is no second datastore to wire up. The LLM runs **on-device inside the app** (Gemma 4 E4B) — there is no LLM server to run, no API to pay for, and a dead venue wifi cannot touch it. Photos go to a folder on disk. Everything starts with one docker compose up plus two dev servers.

What this costs: ₹0. Every piece is free and open source, and nothing runs in the cloud. The model runs on-device in the app; Postgres, FastAPI and the dashboard run on **your own laptop** under Docker; maps use OpenStreetMap, not Google. No API key, no server rental, no subscription anywhere in the stack. You can build and demo the whole thing without spending a rupee.

3. Tech stack

with the reasoning

LAYER	CHOICE	WHY THIS, FOR A HACKATHON
Dashboard	React 18 + TypeScript, built with Vite. Tailwind CSS. TanStack Query for data. Recharts for charts. React-Leaflet for the map.	Vite starts in under a second. Tailwind means no CSS files to argue about at 3am. Leaflet is free and needs no API key — Google Maps would need billing set up.
Mobile	React Native via Expo . expo-camera, expo-location, expo-sqlite, expo-crypto, expo-file-system.	Expo gives you camera, GPS, SQLite and crypto without writing native code. You will prebuild to a real Android project for the on-device model — that is expected, and you keep every Expo module when you do.
Backend	Python 3.11, FastAPI, Uvicorn, SQLAlchemy 2, Pydantic v2, Alembic.	One language for API and ML means no serialisation layer between them. FastAPI generates interactive API docs at /docs free, which is itself demoable.
Database	PostgreSQL 16 with PostGIS and pgvector extensions, in one container.	Spatial queries and vector similarity in the same SQL. No Pinecone account, no separate geo service.
Embeddings	sentence-transformers, model all-MiniLM-L6-v2 .	80 MB, runs on CPU, good enough for 50 clauses. Downloads once — do it before you travel.
LLM	Gemma 4 E4B , on-device only, via LiteRT-LM inside the app. No Ollama, no server, no cloud, no API.	The model ships inside the app and runs on the phone or the arm64 emulator — it cannot be rate-limited, cannot go down, and costs nothing to run. One place the model lives means one set of prompts and one tool schema, with nothing to keep in sync. Apache-family license, free for commercial use.
Risk model	XGBoost via scikit-learn, trained on synthetic history.	Trains in seconds on a laptop. Be upfront that the data is synthetic — see section 7.
Vision	Ultralytics YOLOv8n, pretrained. No training.	Detects people and objects out of the box. Enough to demo “photo screened before an officer sees it”.
Sensors	A Python script posting to the API every 2 seconds. MQTT via Mosquitto only if you have spare time.	MQTT looks impressive and adds a failure point. The demo is identical either way.
Auth	JWT with python-jose, passlib for hashing. Three seeded roles.	Mine Manager, Safety Officer, Regulator. Enough to show role-based views.

4. What to install

do all of this before the hackathon

Machine prerequisites

TOOL	VERSION	NOTE
Node.js	20 LTS	Install via nvm so you can switch if a package complains.
Python	3.11	Not 3.13. Some ML wheels lag behind the newest Python.
Docker Desktop	current	Runs Postgres. Enable WSL2 backend on Windows.
Git	current	One repo, feature branches, merge often.
No LLM server	—	You install no Ollama and no model server. The one model (Gemma 4 E4B) ships inside the app and runs on-device via LiteRT-LM. Nothing to pull on the laptop, nothing to pay for. See section 8 for the on-device install.
Android Studio	current	Your build and debug environment. Install the Android SDK, platform-tools and a JDK 21 toolchain through it. adb comes with platform-tools — put it on your PATH.
Expo Go	from Play Store	Useful early, on two team phones, for everything that is not the on-device model. The model needs a native build.

One model, on-device, doing everything. Gemma 4 E4B runs inside the app and handles all the language work — and, because it is multimodal, the image and voice work too. There is no server model, no second deployment, nothing to keep in sync. The hardest task is **rulebook extraction**; keep it reliable not by swapping models but by **grounding** — retrieve the clause first with MiniLM, pass it into the prompt, and require the citation in the output. A grounded E4B is accurate enough for clause extraction; an ungrounded one of any size will invent regulation numbers.

Backend dependencies

```
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate

pip install fastapi uvicorn[standard] sqlalchemy alembic psycopg2-binary \
    pydantic pydantic-settings python-multipart \
    python-jose[cryptography] passlib[bcrypt] \
    sentence-transformers pgvector \
    scikit-learn xgboost pandas numpy \
    ultralytics pillow \
    shapely geoalchemy2 \
    httpx apscheduler python-dotenv

pip freeze > requirements.txt
```

Dashboard dependencies

```
npm create vite@latest web -- --template react-ts
cd web
npm install
npm install -D tailwindcss @tailwindcss/postcss postcss autoprefixer
npm install @tanstack/react-query axios react-router-dom \
    recharts leaflet react-leaflet date-fns lucide-react
```

Mobile dependencies

```

npx create-expo-app mobile
cd mobile
npx expo install expo-camera expo-location expo-sqlite \
    expo-file-system expo-crypto expo-image-manipulator \
    @react-native-async-storage/async-storage
npm install axios date-fns

# on-device LLM – then add its config plugin to app.json
npm install react-native-litert-lm react-native-nitro-modules

npx expo prebuild          # generates android/ and ios/
npx expo run:android       # builds onto the USB-connected S24+

```

Pin the version. Install the LLM binding once, note the exact version npm resolved, and change ^ to an exact pin in package.json. A minor bump to a native module mid-hackathon means a fresh Gradle build and a new class of bug at the worst hour.

Database container

You need PostGIS and pgvector together. Build a tiny image once:

```

# infra/Dockerfile.db
FROM postgis/postgis:16-3.4
RUN apt-get update && apt-get install -y postgresql-16-pgvector \
    && rm -rf /var/lib/apt/lists/*

```

```

# infra/init.sql
CREATE EXTENSION IF NOT EXISTS postgis;
CREATE EXTENSION IF NOT EXISTS vector;

```

```

# docker-compose.yml
services:
  db:
    build: { context: ./infra, dockerfile: Dockerfile.db }
    environment:
      POSTGRES_USER: anupalan
      POSTGRES_PASSWORD: anupalan
      POSTGRES_DB: anupalan
    ports: ["5432:5432"]
    volumes:
      - ./infra/init.sql:/docker-entrypoint-initdb.d/init.sql
      - pgdata:/var/lib/postgresql/data
volumes: { pgdata: {} }

```

Verify before you travel. Run the compose file, connect with DBeaver, and confirm both `SELECT postgis_version();` and `SELECT '[1,2,3]'::vector;` return without error. Debugging an extension at the venue is a wasted hour.

5. Repository structure

```

anupalan/
├── docker-compose.yml
├── infra/           Dockerfile.db · init.sql
├── api/
│   ├── main.py      FastAPI app, CORS, routers
│   ├── config.py    env settings
│   ├── db.py        engine, session
│   ├── models.py    SQLAlchemy tables
│   ├── schemas.py   Pydantic in/out
│   ├── auth.py      JWT, roles
│   └── routers/
│       ├── obligations.py  ledger CRUD, filters
│       ├── evidence.py    upload, hash chain, vision
│       ├── sensors.py     ingest readings, list alerts
│       ├── rulebook.py    paste clause -> duties
│       ├── risk.py        score per mine and clause
│       ├── geo.py         lease polygons, breach check
│       └── returns.py     draft, sign, lock
│   └── services/
│       ├── llm.py         prompt templates + tool schema (E4B runs on-device)
│       ├── embeddings.py  MiniLM encode + pgvector search
│       ├── risk_model.py  train, predict, feature build
│       ├── hazard.py      rolling window + z-score
│       └── vision.py      YOLOv8n inference
│   └── seed/
│       ├── clauses.json   50 statutory duties
│       ├── mines.json     3 mines + lease polygons
│       └── seed.py        loads everything, trains model
│   └── storage/           evidence photos land here
├── web/                  React dashboard
│   └── src/ pages · components · api · hooks
├── mobile/               Expo app
│   └── app/ screens · db (sqlite queue) · sync.ts
├── tools/
│   └── sensor_sim.py      pushes readings every 2 seconds

```

6. Data model

eight tables, that is all

TABLE	KEY COLUMNS
mine	id, name, subsidiary, type, lease_geom GEOMETRY(Polygon,4326)
statute	id, act, clause_ref, text, embedding VECTOR(384), version, ingested_at
obligation	id, statute_id, mine_id, title, owner_role, frequency, due_date, evidence_type, status
evidence	id, obligation_id, photo_path, lat, lon, captured_at, inside_lease BOOL, vision_result JSONB, prev_hash, chain_hash
sensor_reading	id, mine_id, sensor_type, value, unit, recorded_at
alert	id, mine_id, obligation_id, severity, message, source, created_at, acknowledged_by
risk_score	id, mine_id, obligation_id, score, features JSONB, computed_at
statutory_return	id, mine_id, period, draft_json, signed_by, signed_at, locked BOOL

The hash chain, concretely

```
chain_hash = sha256(prev_hash + photo_sha256 + lat + lon + captured_at + obligation_id)
```

Store prev_hash as the previous evidence row's chain_hash for that mine. A verification endpoint walks the chain and returns OK or the first broken link. That endpoint *is* the demo — edit one row in DBeaver, re-run it, watch it fail.

7. Building the five AI pieces

hours, not weeks

Which model does what — the whole map on one page

There are **four** models in ANUPALAN, not one, and two features that deliberately use no model at all. Knowing which is which is the difference between an architecture you can defend and a black box.

FEATURE	WHAT DOES THE WORK	RUNS ON	WHY THIS ONE
1. Digital Rulebook clause → tracked duty	MiniLM retrieves the right clauses, then Gemma 4 E4B extracts the structured duty	Laptop	Two steps on purpose. Embedding finds the relevant text cheaply; the LLM only ever reasons over text you handed it, never from memory.
2. Risk engine which duty fails next	XGBoost	Laptop	A number an auditor can challenge and you can explain feature by feature. Never an LLM.
3. Hazard early-warning sensor breach detection	No model. Rolling mean + z-score against a threshold table. E4B only explains the alert afterwards	Laptop	A statutory safety alert cannot depend on a probabilistic system. Detection is arithmetic; only the wording is AI.
4. Geo-compliance outside the lease?	No model. PostGIS <code>ST_Contains</code>	Postgres	Geometry is a fact, not an opinion. Reaching for a model here would be a mistake a judge would spot.
5. Vision triage evidence photos	YOLOv8n detects; E4B describes	YOLO: laptop E4B: on-device	Different jobs. YOLO gives you a countable, thresholdable detection — “person, no helmet, 0.91”. E4B, on the phone, gives you a sentence about what is wrong with the ventilation duct. (YOLOv8n is AGPL — fine for the hackathon demo; swap for NanoDet or E4B-vision only if you ever ship it closed-source.)
6. Field app voice, chat, offline drafts	Gemma 4 E4B , entirely offline	Phone / emulator	The one model, doing all language, vision and voice on-device — no server anywhere. Runs the same on the arm64 emulator and the S24+.

MODEL	SIZE	WHAT IT IS	ITS ONE JOB
Gemma 4 E4B	3.66 GB on-device	8B params, 4.5B effective. Text + image + audio.	All language work. Clause extraction, voice to structured observation, ledger questions, report drafting, explaining a score, chat.
all-MiniLM-L6-v2	90 MB	Sentence embedding, 22M params	Turns statute text into vectors so retrieval finds the right clause. Runs before every rulebook call.
XGBoost	under 1 MB	Gradient-boosted trees	The compliance risk score, plus its top three contributing features.
YOLOv8n	6 MB	Object detection CNN, 3.2M params	People, machinery and helmets in an uploaded evidence photo.

Notice what is missing: there is no speech-to-text model. Most teams building a voice feature bolt on Whisper — another model, another 500 MB to 1.5 GB, another failure point, another thing to load on a phone. **Gemma 4 E4B takes audio natively**, so the officer's spoken Hindi goes straight into the same model that produces the structured observation, in one call. That is one fewer dependency than the obvious architecture, and it is worth saying out loud to a judge.

The sentence that ties it together. “Four models, and we chose deliberately where not to use one. Language work is the LLM. Scoring is gradient boosting, so it is reproducible. Detection and boundary checks are arithmetic and geometry, because a statutory alert cannot depend on a probabilistic system.” That answer pre-empts the two questions every AI project gets asked: is the AI real, and can you trust it.

Regulation-as-Code

- Seed `clauses.json` with 50 real duties drawn from the Mines Act 1952, CMR 2017 and the Mines Rules 1955. Your domain research document has the specific regulation numbers — Reg. 43, Reg. 46, Rule 29-P and so on.
- On startup, embed each clause with MiniLM and store the vector in `statute.embedding`.

- The live demo: paste circular text into a box. Chunk it, embed it, retrieve the nearest existing clauses for context, then prompt the LLM for strict JSON: {title, owner_role, frequency, evidence_type, clause_ref}.
- **Force the citation.** Reject any LLM response without a clause_ref. That rule is your answer to the liability question, so make it real in code.

Compliance risk engine

- Generate roughly 2,000 synthetic rows: days overdue, count of past violations, days since last inspection, hazard readings, season, mine type. Label = did it breach.
- Train XGBoost in seed.py. It takes seconds. Persist with joblib.
- Display the score with its top three contributing features so it reads as reasoning, not a magic number.

Say the word “synthetic” before a judge asks. “This model is trained on synthetic data shaped like CIL and DGMS inspection history, because that data is not public. The feature set and the pipeline are real; only the rows are generated.” Teams get caught trying to hide this. Stating it first reads as rigour.

Hazard early-warning

- sensor_sim.py posts methane, water level, strata convergence and vibration every 2 seconds, with a keypress to inject a spike on cue.
- Detection is a rolling mean plus z-score against a static threshold table. Do not reach for an LSTM.
- Each alert must name the clause it threatens. That link is what makes it a *compliance* alert rather than a generic sensor dashboard.

Geo-compliance

- Store three mine lease polygons as GeoJSON, loaded into PostGIS.
- Breach check is one query: `SELECT ST_Contains(lease_geom, ST_SetSRID(ST_Point(:lon, :lat), 4326))`.
- For the excavation demo, hard-code a polygon that crosses the boundary and use ST_Difference to compute the area outside. Render both on Leaflet.
- The mobile app calls the same check on capture, which is how a photo gets its inside_lease flag.

Vision triage

- YOLOv8n pretrained on COCO detects people and equipment with no training at all.
- For helmets, pull a pretrained hard-hat model from Hugging Face rather than training one. If nothing works, scope the claim down to “detects people and machinery in frame” and stay honest.
- Run inference on upload, store the result in evidence.vision_result, and surface it as pass or flag on the dashboard.

8. The offline model

Gemma 4 E4B — the answer, and why

Use **Gemma 4 E4B**. One model, one file — `gemma-4-E4B-it.litertlm`, **3.66 GB** — running through **LiteRT-LM**, which is Google's own production on-device runtime. It does every task on your list, it fits an 8 GB phone with room to spare, and it is **Apache 2.0**, so you can ship it commercially with no licence problem. **That is the whole answer. Everything below is detail you can check.**

Does it actually do what you asked for?

WHAT YOU WANT	HOW THE MODEL DOES IT
Collect sensor data	Function calling. You declare a tool — <code>get_sensor_readings(mine_id, window)</code> — and the model calls it, then receives typed JSON back. It does not read the sensor itself; it decides <i>when</i> to look and <i>what</i> to ask for. That is the part worth automating.
Make a chart	Same mechanism. The model returns <code>{chart_type, x, y, title, caption}</code> and your React Native code draws it. Never ask a language model to draw — ask it to specify. That is the difference between a demo that works every time and one that works sometimes.
Chat and help	Native, and in 140+ languages — Hindi, Bengali, Odia, Telugu, the languages your users actually speak underground.
Analyse a picture	Native image input, one call: <code>sendMessageWithImage(text, path)</code> . “Is this ventilation duct damaged?” against a photo taken thirty seconds earlier.
“Is this critical?”	69.4% on MMLU Pro with configurable thinking. Give it the readings and the governing clause; it returns a severity with its reasoning shown. Turn thinking on for this, off for a lookup.
Draft a report	Its single strongest task. Evidence in, structured draft out, every figure traceable.
Features you add later	128K context, tool calling, text + image + audio. A new capability is a new tool declaration , not a new model. This is the part that makes it worth building on.

Will it run on the phones you care about?

Your target is a good post-2024 Android phone: 8 GB RAM, 256 GB storage, a real processor. E4B fits that comfortably — and how comfortably depends on which backend the phone can give you.

BACKEND	MEMORY USED	SPEED	AVAILABLE ON
GPU (OpenCL)	710 MB	~22 tok/s	Qualcomm Snapdragon (Adreno), Samsung (Xclipse), most MediaTek. This is the good case — the model barely touches your RAM budget.
CPU	3.3 GB	~18 tok/s	Everything. Your guaranteed floor. Even this fits an 8 GB phone.
NPU	varies	fastest	Chip-specific prebuilt files. Google publishes them for Google Tensor G5 and G6, Snapdragon 8 Elite, and Qualcomm QCS8275 — which is the industrial chip inside rugged field handhelds.

Ship it like this: request GPU at load, fall back to CPU automatically. Worst case on any 8 GB phone is 3.3 GB and 18 tokens per second, which still works. Best case is 710 MB, and the phone barely notices the model is there. **You do not have to choose in advance — the runtime picks per device.**

Those are measured numbers, not my estimates. Google publishes E4B on LiteRT-LM benchmarked on a Galaxy S26 Ultra: GPU backend 1,293 tok/s prefill, 22.1 tok/s decode, 0.8 s to first token, 710 MB; CPU backend 17.7 tok/s, 3,283 MB. Speculative decoding raises GPU decode to **36.7–49.4 tok/s**. A good 2024 phone will land somewhat below the S26 Ultra, but the shape holds.

Two things about testing that will cost you hours if nobody tells you

1. **The DEFAULT emulator image will not run the model.** The standard Pixel emulator is `x86_64` — it cannot load arm64 native code, and `loadModel` will crash silently. Every other part of your app runs fine in that image; only the LiteRT-LM binding fails.
2. **A real Pixel is the worst phone for the GPU backend.** Google's own Tensor chips do not expose OpenCL. There is an open issue on the LiteRT-LM repository titled, almost word for word, “*Backend.GPU() silently fails on Pixel 8 Pro — OpenCL not found*”. On a Pixel you get the CPU backend only — not the 710 MB path.

To run the model in the Android Studio emulator: download an arm64-v8a system image.
AVD Manager → Create Virtual Device → Pixel 7 → System Image → **Other Images** tab → filter ABI = **arm64-v8a** → download Android 14 or 15.

The arm64 image runs via binary translation on your x86 laptop. The model **will** load and answer, but expect **2–4 tokens/second** instead of the 22+ tok/s you get on a real phone. Functional for development; slow for a live demo. The GPU backend is also unavailable in the emulator regardless of image — you will get the CPU path (3.3 GB, slowest, but it works).

Use both: arm64 emulator for daily development and UI iteration (no cable needed, easy to reset), physical S24+ over USB for any demo that involves the model performing in real time. Switching between them is just picking a different run target in Android Studio — zero code changes.

Why not something bigger

Gemma 4 **12B** is the better model on paper — 77.2% on MMLU Pro against E4B's 69.4%. It is not a phone model, and the clearest evidence is Google's own: the LiteRT-LM release gives E4B an **Android benchmark table, an iOS table and Play Store links**, and gives the 12B **desktop and web only**, at a measured 7.7–8.0 GB. It is also dense, so it streams ~7 GB of weights per token where E4B streams 2.24 GB — roughly three times slower before you even reach the memory problem. **Do not spend time on it.**

The one fallback, for phones smaller than your floor

Gemma 4 E2B — 2.59 GB, same family, same code, same API, same text + image + audio. If you ever need to run below 8 GB, read `ActivityManager.MemoryInfo` at first launch and fetch E2B instead of E4B. **This is a safety net, not a decision you make now.** Build against E4B and forget E2B exists until a device forces it.

How to install it

```
npm install react-native-litert-lm react-native-nitro-modules
# add its config plugin to app.json, then:
npx expo prebuild           # generates android/ and ios/
npx expo run:android        # builds onto the USB-connected S24+

# push the model once, over the cable – never over venue wifi:
adb push gemma-4-E4B-it.litertlm /sdcard/Download/
```

Load it with the local path and request the GPU: `loadModel(path, { backend: "gpu" })`, streaming through `sendMessageAsync`. Then open the generated `android/` folder in Android Studio for logcat, the memory profiler and native crashes.

Pin the version. Note the exact version npm resolves and replace the `^` with an exact pin in `package.json`. Also: there is a known crash in the underlying `litertlm-android 0.15/0.16` library where passing a `suppressTokens` option aborts the process on Android. You have no reason to use that option — just don't.

One model, on-device only — no server

There is one deployment of Gemma 4 E4B and it lives **inside the app**, running through LiteRT-LM on the device. No laptop LLM, no Ollama, no cloud, no API to pay for. The same `.litertlm` file runs on the arm64 emulator and on your S24+ — the runtime just picks a different backend underneath.

WHERE	BACKEND	SPEED	ROLE
Emulator (arm64-v8a)	CPU	~2–4 tok/s	Development and UI, on your laptop screen. Also fine for demoing the language moments if no phone is to hand.
S24+ (physical)	GPU	~22 tok/s	The real demo device. Airplane mode, officer speaks Hindi, the model answers entirely on the handset.

No routing, no env switch, no fallback to manage. Because the model only ever runs on-device, there is a single code path: `sendMessageAsync` from the LiteRT-LM binding. One prompt set, one tool schema, one place the model lives. The free local FastAPI + Postgres backend on your laptop stores what the app produces and serves the dashboard — it does **no LLM work at all**, so nothing about the intelligence depends on a network or a paid service.

The trade you are making. On-device is now your only path, so there is no laptop model to fall back to if the native build misbehaves at the venue. Buy that safety back three ways: validate on the S24+ tonight with Google AI Edge Gallery (real E4B, airplane mode, zero code); do the native build **before** you travel; and record an OBS video of it working the night before. Do those and on-device-only is a strength, not a risk.

How Android Studio fits into the demo

Android Studio is your **build and deploy tool**, not an emulator. Plug the S24+ in over USB, enable USB Debugging on the phone, then:

```
npx expo run:android          # Android Studio compiles and installs the APK on the S24+
                              # The model runs on the phone's GPU – Android Studio just controls it
```

While the demo runs, Android Studio shows you **Logcat** (every token the model streams, every tool call), the **Memory Profiler** (3.66 GB mapped, GPU usage), and the **native crash reporter** if something goes wrong. The judges see the phone screen through scrcpy. You see everything behind it in Android Studio.

This is the demo setup. S24+ on a USB cable, scrcpy mirroring to the projector, Android Studio open on your laptop for visibility. One cable does the APK install, the model push, the screen mirror, and the debugger — all at once.

What the model can genuinely own — and what it must not

You want to show the model doing real work rather than decorating the app. It can, and here is the honest split. Put this on a slide; the division itself is an engineering argument in your favour.

THE MODEL OWNS THIS	DETERMINISTIC CODE OWNS THIS
Clause to duty. Statutory text in, {title, owner_role, frequency, evidence_type, clause_ref} out. The whole Regulation-as-Code feature is the model.	The risk score. XGBoost. Reproducible, inspectable, defensible to an auditor.
Voice or free text to structured observation. Officer speaks; model returns hazard type, severity, affected clause.	Threshold breach detection. A statutory safety alert cannot depend on a model's mood.
Reading a sensor window and explaining it. Given readings plus the relevant clause: what is breaching, why it matters, what the duty requires. This is where sensor data actually meets the model — interpretation, not arithmetic.	Boundary containment. PostGIS ST_Contains. A geometric fact, not an opinion.
Question answering over the ledger. “What is overdue in panel 3B and who owns it?”	Hash-chain verification. Cryptography, verified or not.
Drafting the statutory return from collected evidence, every figure cited.	Filing anything. A certificated officer signs. Always.
Explaining the risk score in plain language to the officer who has to act on it.	Computing that score. The model narrates it; it does not decide it.

That left column is comfortably 40–50% of the cognitive work in the product, and all of the user-facing intelligence. Say the split out loud to a judge: “The model does the language work. Deterministic code does the safety-critical work. A statutory alert cannot depend on a probabilistic system, so we drew the line deliberately.” That answer is stronger than claiming the AI does everything, and it is the answer an engineer at Coal India would want to hear.

How to make the claim visible. Colour-code the architecture diagram in your finale deck — amber for model-driven, blue for deterministic. A judge can then see the proportion rather than take your word for it, and it pre-empts the “is the AI actually doing anything?” question before it is asked.

How to ship it without risking the demo

- **Validate it tonight, before writing any code.** Install **Google AI Edge Gallery** from the Play Store on the S24+, download Gemma 4 E4B inside it, switch to airplane mode and use it. Same model, same runtime, zero code, fifteen minutes — and you will know what your model actually does on your hardware, including whether the audio path holds, before you spend an hour of the 36 on it.
- **Do the native build before you travel.** Prebuild, add the binding, get one hard-coded prompt answering on the S24+. Gradle resolving a new native dependency for the first time on venue wifi is how teams lose an evening.

- **Build the app skeleton first, wire the model in last.** Get the app, dashboard and backend talking end to end before you add the on-device model. Bring E4B in around hours 24–30 once the plumbing holds — but treat it as core, not optional, because it is now your only LLM.
- **Warm the model behind a splash screen.** Mapping 3.66 GB takes a few seconds on first open. Never in front of a judge.
- **Ground it, always.** Retrieve the clause first, pass it in, require the citation. A 4B model asked to recall statute from memory will invent regulation numbers, and a reviewer who knows the Mines Act will catch it in one question.
- **Cap output at 150 tokens.** Tight prompts, short answers. Long generation is where on-device models feel slow.

The demo to aim for. Phone in airplane mode. The officer holds it up and says, in Hindi, “gas reading is high in panel three”. The model — running entirely on the handset, no network — transcribes it, maps it to Regulation 46, drafts the observation with the clause cited, and drops it in the sync queue. That is a thirty-second sequence no other team will have, and every part of it is something your hardware can do today.

9. Software you need for live testing

TOOL	FOR	NOTE
Android Studio + adb	Building and running on the S24+	Your primary loop. <code>npx expo run:android</code> with the phone on USB; Android Studio for logcat, the memory profiler and native crashes. The emulator cannot run the on-device model — arm64 only.
Expo Go	Quick UI iteration only	Still fine for screens that do not touch the model. Scan the QR from <code>npx expo start</code> ; phone and laptop must share a network.
scrcpy	Mirroring the phone to the projector	Free, no root, works over the same USB cable. This is how you demo a phone to a room — far better than holding it up or pointing a camera at it. Install and rehearse it.
Cloudflare Tunnel or ngrok	Phone reaching your API	Venue wifi usually blocks device-to-device. <code>cloudflared tunnel --url http://localhost:8000</code> gives a public HTTPS URL in seconds and has no signup.
Bruno or Postman	Testing endpoints	Bruno stores collections as files in your repo, so the whole team shares them through git.
DBeaver or pgAdmin	Inspecting the database	Also how you demo the hash chain breaking — edit a row live.
FastAPI /docs	Free API console	Built in at <code>localhost:8000/docs</code> . Judges like seeing it.
Chrome DevTools	Responsive checks, throttling	Network tab set to Offline is a second way to demo offline behaviour.
Airplane mode	The offline demo	The real one. Rehearse it — know exactly how long sync takes.
OBS Studio	Fallback recording	Record all five demo moments working, the night before. If anything dies on stage you still have a demo.
MQTT Explorer	Only if you use MQTT	Pairs with Mosquitto in Docker. Optional.
Vercel + Render	Public URLs, if wanted	Free tiers. Render cold-starts take about a minute, so keep local as the primary demo path.

10. Who builds what

six people, four tracks

TRACK	PEOPLE	OWNS
Backend core	2	FastAPI skeleton, models, auth, obligations, evidence upload, hash chain, returns. Owns the database schema — all migrations go through them.
AI core	2	llm.py, embeddings, rulebook extraction, risk model, hazard detection, vision. Works against the API contract, not the database directly.
Dashboard	1	Ledger table, KPI row, alerts panel, risk bars, Leaflet map, return sign-off screen.
Mobile	1	Duty list, camera capture, SQLite queue, sync with conflict handling, offline banner.

Agree the API contract in hour one, before anyone writes a feature. Write the endpoint paths and JSON shapes on a whiteboard, then have each track mock the others. This is the single thing that decides whether integration at hour 30 takes twenty minutes or destroys your night.

11. The 36 hours

HOURS	MILESTONE — nothing moves on until this is true
0–2	Repo created, compose up, database live with both extensions, API contract agreed and written down.
2–6	Schema migrated. Seed data loaded: 50 clauses embedded, 3 mines with lease polygons, 3 users. Auth returns a token.
6–12	Ledger endpoints live and rendering in the dashboard. Mobile shows the duty list from the real API.
12–18	Evidence capture works: photo, GPS, hash chain, upload. Sensor simulator streaming. Checkpoint: demo moment 2 works end to end.
18–24	Rulebook extraction returns valid cited JSON. Risk model trained and scoring. Alerts firing from the simulator.
24–30	Map with breach detection. Vision on upload. Return drafting and sign-off. Checkpoint: all five moments work, however roughly.
30–34	Freeze features. Polish only. Seed a believable demo dataset. Record the OBS fallback video.
34–36	Three full dry runs against the clock. Fix only what breaks during them.

Feature freeze at hour 30 is not a suggestion. Every team that loses does so by adding one more thing at hour 33 and breaking the demo. Nothing new after 30 — only fixes to things that failed a dry run.

12. When things break at the venue

FAILURE	WHAT YOU DO
Wifi is down or blocked	The model is on the phone; Postgres and the dashboard are on your laptop — all local. The on-device app needs no network at all; only the dashboard-to-phone data sync uses the free tunnel. Airplane mode is the demo, not the failure.
Phone will not reach the API	Laptop hotspot. Set it up and test it once before you need it.
LLM returns malformed JSON	Retry twice, then fall back to a keyword-rule extractor. Ugly, invisible to the judge, keeps the demo alive.
Everything is broken	Play the OBS recording and talk over it. A team with a recording looks prepared. A team with a black screen does not.

13. Before you travel — checklist

- Docker image built and the database starting cleanly with both extensions confirmed.
- Gemma 4 E4B validated on the S24+ in Google AI Edge Gallery (airplane mode) and answering a test prompt.
- MiniLM and YOLOv8n weights downloaded and cached — both fetch from the internet on first use.
- Android Studio installed, adb devices listing the S24+ over USB, and npx expo run:android putting a hello-world build on it.
- **Google AI Edge Gallery installed on the S24+ with Gemma 4 E4B downloaded and answering in airplane mode.** Fifteen minutes, and it settles whether the whole on-device feature is real before you build on it.
- The .litertlm file pushed to the phone with adb push, and scrcpy mirroring the S24+ to a laptop screen.
- Cloudflare Tunnel or ngrok installed and tested once.
- The 50 clauses written into clauses.json. Do this at home; it is research, not coding, and doing it on the clock is a waste of your 36 hours.
- Every dependency in this document installed on every laptop, not just one.
- Repo pushed, everyone can clone and run it.

The clause file is the highest-value thing you can prepare in advance. Fifty real duties with correct regulation references, owners, frequencies and evidence types. It costs you an evening now and saves you four hours at the venue — and it is the difference between a demo that mines actual statutory language and one that mines lorem ipsum.